LLM-Driven Evolution of L2 Cache Prefetchers in ChampSim

15-740 Final Project Report

0. Abstract

Hardware prefetcher design exposes a large hyper-parameter space: classification thresholds, prefetch degrees, scoring formulas, and the control-flow of the classifier itself. We apply OpenEvolve, an LLM-in-the-loop evolutionary search system, to three production-grade L2 prefetchers in ChampSim — the Best-Offset Prefetcher (BOP), the IP-Classifier Prefetcher (IPCP), and Berti — and use a 90:10 IPC-versus-accuracy fitness function on `602.gcc_s` as the search signal. Twenty iterations per prefetcher discovers configurations that lift gcc IPC by 27–34% over the as-shipped versions and yield combined scores of 2.08 (BOP), 2.18 (IPCP), and 2.20 (Berti). The IPCP gain comes from a structural mutation — making `classify_ip()` non-static so it can read `global_stream_dir` — found at iteration 1 and never exceeded. As an additional line of improvement we then port the evaluator to a multi-trace fitness signal across an eight-trace SPEC CPU 2017 subset and run a further 58 candidates on IPCP; the resulting `IPCP-v40` configuration lifts the multi-trace combined score from 1.243 to 1.316 (+5.9%) and the geometric-mean speedup from 1.293× to 1.372×, with no per-trace regression. The multi-trace finding is methodological rather than mechanical: the single-trace optimum was a local optimum on gcc, and a single hyper-parameter (`GS_DEGREE`) carries the bulk of the headroom the gcc-only signal could not see.

1. Introduction

Hardware prefetchers hide DRAM latency by predicting future memory accesses. Modern designs expose dozens of numeric thresholds, scoring functions, and classification rules; the design space is large and sensitive to the workload mix used during tuning, and most published configurations are hand-tuned by their authors against a small set of benchmarks. This project asks whether LLM-driven evolutionary search can discover comparable or better configurations automatically.

We use OpenEvolve, an open-source reimplementation of the AlphaEvolve pattern: an LLM proposes diff-based mutations to a marked region of source code, each candidate is compiled and scored by a user-supplied evaluator, and a multi-island genetic algorithm maintains a population. The marked region (`EVOLVE-BLOCK`) gives the LLM tightly scoped write access to the parts of the prefetcher we are willing to vary, and the rest of the simulator is untouched. We target three L2 prefetchers (BOP, IPCP, Berti) and a single SPEC CPU 2017 trace, `602.gcc_s`, with a 90:10 IPC-vs-accuracy fitness function.

A natural question is whether single-trace fitness generalises. As an additional line of improvement we replace the gcc-only evaluator with a multi-trace fitness signal aggregated geometrically across an eight-trace SPEC subset and re-run the IPCP evolution. The resulting v40 configuration both improves the multi-trace score by 5.9% and narrowly tops the single-trace gcc number.

2. Related Work

IPCP (Pakalapati & Panda, ISCA 2020) defines the four-class IP-Classifier prefetcher (CS, CPLX, GS, NL) we evolve. **Best-Offset Prefetcher** (Michaud, HPCA 2016) selects a single prefetch offset by scoring candidate offsets against a recent-access filter. **Berti** (Navarro Torres et al., MICRO 2022) is a timing-aware delta prefetcher that ranks observed strides by how *timely* the implied prefetch would have been against the cache's average miss latency. Reduced ports of all three are available in the `ChampSim` repository (Gober et al., 2022) and we treat their as-shipped configurations as the "stock" baselines.

OpenEvolve is an open-source reimplementation of the **AlphaEvolve** pattern (Romera-Paredes et al., 2024). The LLM is prompted with the current code, a system message describing the fitness function, and the per-candidate score history; it returns a diff that the framework applies, compiles, and re-scores. We use `gpt-5-mini` via the CMU LiteLLM gateway with a population of 30 per island, two islands, 20 iterations, an exploitation ratio of 0.7, and a maximum candidate length of 8000 tokens.

ChampSim is the trace-driven simulator. Our configuration is a 4-GHz OoO core with a 352-entry ROB, 64×8 L1I, 64×12 L1D, 1024×8 L2C (10-cycle), 2048×16 LLC (20-cycle), and DDR4-3200 (t_{CAS}=t_{RCD}=t_{RP}=24). The L2 prefetcher operates on physical block addresses; cross-page prefetches are filtered out by `issue_prefetch`'s page-boundary check.

3. Method

3.1 Evolution setup

For each prefetcher we mark a contiguous region of the C++ source with `// EVOLVE-BLOCK-START` / `// EVOLVE-BLOCK-END`. The region contains the configurable constants, scoring functions, offset lists, and classifier logic that we are willing to vary. The rest of the prefetcher (state schema, hooks called by ChampSim) is fixed. The LLM is given a system message describing the role of each constant and is explicitly told what it may and may not change.

The fitness function on `602.gcc_s` (200K warmup + 2M sim instructions) is

$$\text{score} \;=\; 0.9 \cdot \frac{\text{IPC}}{0.471} \;+\; 0.1 \cdot \frac{\text{useful}}{\text{useful} + \text{useless}}$$

where 0.471 is the no-prefetcher gcc IPC. A build or run failure returns score 0; the genetic algorithm's archive ensures the best candidate is preserved across iterations. Each iteration is one LLM call (≈ 30 s) plus one ChampSim build (≈ 60 s) plus one simulation (≈ 30 s).

3.2 Multi-trace extension

After the single-trace evolutions completed, we ported the evaluator to an eight-trace SPEC CPU 2017 subset — perlbench_s, gcc_s, mcf_s, lbm_s, omnetpp_s, xalancbmk_s, deepsjeng_s, xz_s (one region each, ≈3.6 GiB total) spanning branch-heavy, memory-bound, and streaming behaviour. The fitness function aggregates per-trace speedups and accuracies as

$$\text{combined\_score} \;=\; 0.9 \cdot \text{geomean}_T(\text{speedup}_T) \;+\; 0.1 \cdot \text{mean}_T(\text{accuracy}_T)$$

where $\text{speedup}_T = \text{IPC}_T / \text{IPC}_T^{\text{no-pf}}$ is computed against a per-trace no-prefetcher baseline cached in `baselines_ipcp_multi.json`. The geometric mean prevents one strong trace from masking weakness on another. The 0.9:0.1 weighting is unchanged so single-trace numbers under either evaluator are directly comparable when restricted to gcc. We re-evaluated IPCP on the multi-trace evaluator across 58 candidates (`v1`–`v58`) to test whether additional headroom remained beyond the single-trace optimum.

4. Experimental Results

4.1 Single-trace results on `602.gcc_s`

Prefetcher	IPC	Speedup	Accuracy	Combined
no prefetcher	0.471	1.00×	—	—
BOP (stock)	0.821	1.74×	98.7%	1.66
IPCP (stock)	0.811	1.72×	98.7%	1.64
Berti (stock)	0.956	2.03×	96.8%	1.92
BOP (evolved, iter 6)	**1.040**	**2.21×**	96.5%	**2.08**
IPCP (evolved, iter 1)	**1.088**	**2.31×**	97.4%	**2.18**
Berti (evolved, iter 15)	**1.100**	**2.34×**	94.7%	**2.20**

All three prefetchers improve on their hand-tuned stock configurations. The relative IPC gains over stock are BOP +27%, IPCP +34%, and Berti +15%. Single-trace evolution saturates quickly: BOP plateaued at iteration 6 of 20, IPCP at iteration 1, and Berti at iteration 15.

What the LLM changed. For BOP, the mutation doubled `PREFETCH_DEGREE` (1→2), tightened `MSHR_THRESHOLD` (0.7→0.6), shortened `ROUND_MAX` (100→64), and duplicated the small offsets inside `OFFSET_LIST` so good candidates received multiple independent trials. For Berti, the mutation rebalanced

the scoring weights (`TIMELINESS_WEIGHT` 1.0→1.8, `ACCURACY_WEIGHT` 0.5→1.6), introduced a non-linear late-penalty $\text{late\_ratio}^{1.5}$, and added a consistency bonus that scales with $\sqrt{\text{coverage}}$.

The IPCP gain is the most interesting because it is structural rather than numeric. Stock IPCP declares `classify_ip()` as a static member function, so the classifier sees only its four explicit arguments (the IP-local stride history) and cannot read the workload-wide `global_stream_dir` signal that `prefetcher_cache_operate` maintains one scope up. The iteration-1 mutation removed the `static` keyword on both the declaration and the definition and added a stream-intercept block:

```
if (global_stream_dir != 0) {
  if ((global_stream_dir > 0 && new_stride > 0) ||
      (global_stream_dir < 0 && new_stride < 0)) {
    if (confidence < CS_CONFIDENCE_THRESHOLD || old_class == GS)
      return GS;
  }
}
```

This routes IPs that happen to be moving in the global workload's direction — but have not yet accumulated the confidence threshold for the CS class — straight to the GS class, which issues a 3-deep ±1 prefetch chain. On `gcc_s` this re-classifies a large population of "going-with-the-flow" PCs from NL (one prefetch) to GS (three prefetches) and lifts IPC from 0.811 to 1.088.

4.2 Multi-trace extension: IPCP-v40

The single-trace results above leave open whether the IPC gains generalise. We re-ran the evolved BOP, IPCP, and Berti binaries on the eight-trace SPEC subset against a no-prefetcher baseline:

Prefetcher	Geomean speedup	Mean accuracy	Combined
no prefetcher	1.000	—	—
IPCP (stock)	1.195	82.5%	1.158
BOP (evolved)	1.218	82.2%	1.180
IPCP (evolved, iter 1)	1.293	79.1%	1.243
Berti (evolved, iter 15)	1.294	78.0%	1.243
IPCP-v40 (multi-trace)	**1.372**	81.0%	**1.316**

The single-trace-evolved BOP, IPCP, and Berti retain meaningful gains across the multi-trace suite — IPCP's geomean speedup is 1.293× and Berti's is 1.294× — but their combined scores cluster around 1.24, far below their gcc-only 2.0+ figures. Both are catastrophically inaccurate on memory-bound traces: IPCP issues 168K prefetches on `mcf` with only 28.6% accuracy, and Berti issues 166K with 20.9% accuracy. As an additional line of improvement we ran a further 58 candidates on the multi-trace evaluator. The champion (`IPCP-v40`) raises the IPCP combined score from 1.243 to 1.316 (+5.9%) and the geomean speedup to 1.372×.

The per-trace breakdown shows where the gain comes from:

Trace	no-pf	stock IPCP	IPCP iter-1	**v40**	speedup vs. no-pf	Δ IPC vs iter-1
`600.perlbench_s`	1.935	1.986	1.995	1.997	1.03×	+0.002
`602.gcc_s`	0.469	0.806	1.088	1.106	2.36×	+0.018
`605.mcf_s`	0.210	0.270	0.260	0.274	1.31×	+0.014
`619.lbm_s`	0.583	0.615	0.708	0.758	1.30×	+0.050
`620.omnetpp_s`	0.226	0.232	0.230	0.232	1.03×	+0.002
`623.xalancbmk_s`	0.411	0.585	0.739	**1.024**	**2.49×**	**+0.285**
`631.deepsjeng_s`	1.090	1.106	1.109	1.112	1.02×	+0.003
`657.xz_s`	2.156	2.508	2.508	2.508	1.16×	+0.000

`xalancbmk` dominates the gain (+0.285 IPC, +38% over iter-1 IPCP), followed by `lbm` (+0.05). On `gcc` v40 also slightly exceeds the iter-1 IPCP IPC (1.106 vs. 1.088), which means the multi-trace optimum

is also a slightly better gcc-only optimum (combined 2.21 vs. 2.18). On `mcf` the iter-1 IPCP actually regresses below stock (0.260 vs. 0.270): the stream-aware classifier introduced 60K extra useless prefetches there. v40's new `GS_STRIDE_LIMIT`= 8 gate — a one-line addition to `classify_ip()` — recovers the lost `mcf` accuracy while keeping all of the streaming-trace gains.

What the multi-trace evolution changed. v40 differs from the iter-1 IPCP in five constants and one classifier line:

```
constexpr int CS_DEGREE = 8; // was 4
constexpr int CPLX_DEGREE = 8; // was 3
constexpr int GS_DEGREE = 16; // was 3 -- the dominant change
constexpr int GS_STRIDE_LIMIT = 8; // new constant
// CS_CONFIDENCE_THRESHOLD, CPLX_CONFIDENCE_THRESHOLD,
// STREAM_DETECT_THRESHOLD, CONFIDENCE_SAT_MAX, MSHR_THRESHOLD: unchanged

if (dir_match && std::abs(new_stride) <= GS_STRIDE_LIMIT) {
  if (confidence < CS_CONFIDENCE_THRESHOLD || old_class == GS) return GS;
}
```

`GS_DEGREE` carries the bulk of the gain (3→16); the limit is reached because `xalancbmk` absorbs prefetch chains all the way to the 4-KiB page boundary, where `issue_prefetch`'s same-page check silently caps any further degree increase.

4.3 Negative results around v40

We invested 18 further iterations probing structural changes after v40. None beat it by more than measurement noise, and three exposed instructive failure modes. Routing GS prefetches at the IP's actual stride instead of ±1 (`v50`) lost 2.6% — the ±1 chain captures the immediately-adjacent cache line which is hot under spatial locality. Removing the page-boundary check inside `issue_prefetch` (`v54`) dropped `lbm` accuracy from 100% to 72% and regressed combined score to 1.281; ChampSim does propagate cross-page prefetches but every one is marked useless. Boosting confidence by +2 on `useful_prefetch=true` (`v55`) overshot CS classifications and regressed to 1.278. These confirm v40 saturates the local optimum reachable with IPCP's existing state schema and `EVOLVE-BLOCK` surface.

5. Goals and Next Steps

The proposal goal was to demonstrate that LLM-driven evolutionary search can match or beat hand-tuned prefetcher configurations, and to characterise the regime in which it works. We met that goal on the single-trace metric for all three prefetchers (+27 to +34% IPC over hand-tuned), and the multi-trace extension shows that an additional +5.9% combined score is recoverable on IPCP once the fitness signal is the right one.

Three directions are the most promising next steps. First, the page-boundary plateau invites widening the evolvable scope to include `prefetcher_cache_operate` and `issue_prefetch`; with that surface in scope, an evolved prefetcher could compute remaining-lines-in-page dynamically and reallocate the saved degree elsewhere. Second, the `useful_prefetch` and `cache_hit` signals are entirely unused by the current IPCP; a more careful integration — tracking a per-IP rolling accuracy in the IP table rather than perturbing global confidence — could unlock the closed-loop feedback we attempted in `v55`. Third, applying the multi-trace evaluator to BOP and Berti would test whether the single-hyper-parameter pattern (a single dominant degree) generalises beyond IPCP; both currently sit below v40 on the multi-trace suite.

6. Collaboration

Helen implemented the three baselines (BOP, IPCP, Berti). Jerry set up OpenEvolve and ran the single-trace evolutions. Hao ran the multi-trace evolution. All three discussed and reviewed this writeup.

7. Conclusion

LLM-driven evolutionary search discovers prefetcher configurations competitive with hand tuning. Twenty iterations against a single SPEC trace lifted gcc IPC by 27–34% on three prefetchers and surfaced

one structurally meaningful mutation (the non-static `classify_ip()` on IPCP). Re-evaluating those configurations on a multi-trace SPEC subset shows that the single-trace gains do generalise in geomean speedup (1.29–1.31×) but that an additional +5.9% combined score is available on IPCP once the fitness signal is multi-trace. The multi-trace champion (`IPCP-v40`) raises geomean speedup to 1.372× via a single dominant hyper-parameter (`GS_DEGREE` 3→16) and a small magnitude-aware gate that prevents the more aggressive prefetcher from polluting memory-bound traces. The plateau at v40 is enforced by ChampSim's page-boundary check and is therefore an architectural rather than algorithmic limit; further progress requires widening the evolvable region or rethinking IPCP's state schema.